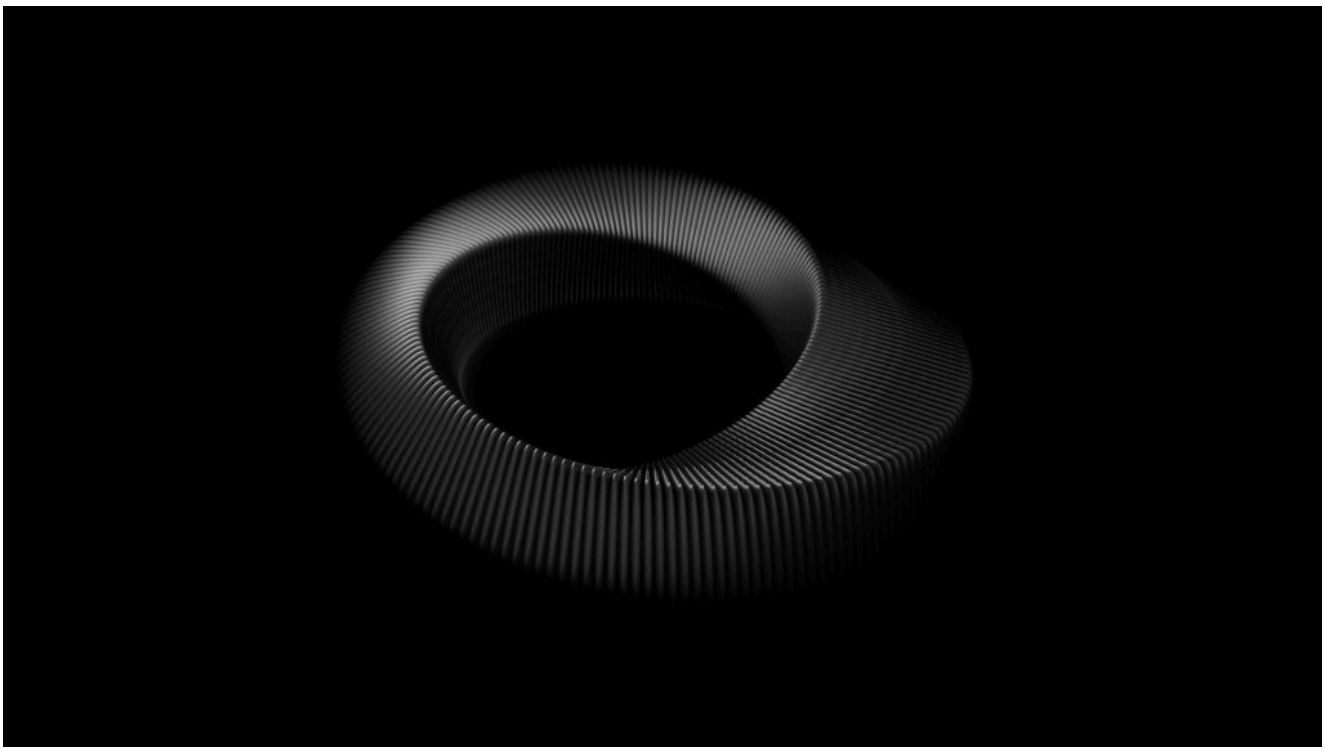
РАЗРАБОТКА ИИ / ИНФРАСТРУКТУРА ИИ / РАЗРАБОТКА ПЛАТФОРМ

Новая задача платформенной инженерии: обслуживание сред со скоростью агента

Агенты искусственного интеллекта для написания кода перегружают рабочие процессы. Узнайте, как сервисная модель масштабирует разработку платформы для ускорения написания кода с помощью ИИ.

18 июля 2026 года, 10:00 Арджун Айер



Вимал С. для Unsplash+

Инженерия платформ одержала верх в этом споре. Около 90% организаций внедрили хотя бы одну внутреннюю платформу; «золотые пути» стали общепринятой практикой, а запросы к среде, на выполнение которых раньше уходили дни, теперь выполняются за часы. По меркам, которые сама дисциплина установила для себя, это победа.

Затем появился самый требовательный клиент, который когда-либо был у тформы, и это был не разработчик. Агент по написанию кода, который хочет проверить свою работу, запрашивает среду так же, как клиент

Организация, в которой 100 разработчиков и каждый из них курирует несколько сеансов работы с агентом в день, к обеду генерирует сотни запросов к среде. Для каждого запроса нужны реалистичные зависимости, и каждый из них становится бесполезным, как только завершается его проверка. Это не очередь заявок. Это трафик.

Самый требовательный клиент, который когда-либо был у платформы

Требование не является умозрительным. По данным Octoverse от GitHub, 43,2 миллиона пул-реквестов объединяются в месяц, что на 23% больше, чем в прошлом году, при этом только агент кодирования Copilot за первые пять месяцев работы открыл более миллиона пул-реквестов. Каждое из этих изменений должно быть протестировано, прежде чем оно будет объединено.

Под этими цифрами меняется структура арендаторов. Опрос Stack Overflow, проведенный в 2025 году, показал, что половина профессиональных разработчиков уже ежедневно используют инструменты на основе ИИ, и каждый из тех, кто пользуется ими ежедневно, может одновременно работать с двумя, тремя или пятью агентами. При оценке потребности в ресурсах больше не учитывается количество сотрудников. Учитывается количество сотрудников, умноженное на количество агентов, умноженное на количество итераций.

Catch Performance Regressions Before the Pull Request With k6 and Signadot Plans
16 July 2026

Comparing Kubernetes Development Tools in 2026: Telepresence, mirrord, Okteto, and Signadot
18 June 2026

> “Coding agents turned environment requests into traffic: concurrent, short-lived, and relentless. The platform teams that keep up will be the ones that stop provisioning environments and start serving them.”

Platform teams can see what is coming. The latest State of Platform Engineering report found that 94% of organizations consider AI critical to platform engineering’s future, and its central theme is the shift from cloud-native platforms to AI-native ones.

What changed is not only the volume but also the shape. Human environment demand is diurnal, negotiable, and tolerant of a morning’s delay. Agents retry, fan out, and iterate in tight loops, and demand that the shape already has a name across the platform. The name is traffic.

TRENDING STORIES

1. Develop like you deploy: closing the Kubernetes local-to-cluster gap

3. You don't have a deployment problem. You have a validation problem.
4. Platform Engineering: What Is It and Who Does It?
5. In 2026, AI Is Merging With Platform Engineering. Are You Ready?

Duplicate everything, and the cost curve kills you

The duplication model hands every request a full copy of the stack. Price one out: a 40-service system with its databases and queues costs a few dollars an hour per copy, takes tens of minutes to assemble, and sits mostly idle during the brief window of validation it exists to support.

Multiply by concurrency, and the model collapses. Hundreds of requests a day with modest overlap means dozens of full copies running at once, and a bill that scales linearly with agent activity. The latency is wrong by an order of magnitude too, because an agent that iterates in seconds cannot wait tens of minutes for its environment to arrive.

Pre-provisioning a warm pool does not rescue the model; it only moves the waste. Agent demand is bursty, so a pool sized for the peak idles through the trough, and a pool sized for the trough queues at the peak. Paying full-copy prices for capacity you mostly do not use is the definition of the wrong cost curve.

Share everything, and the queue kills you

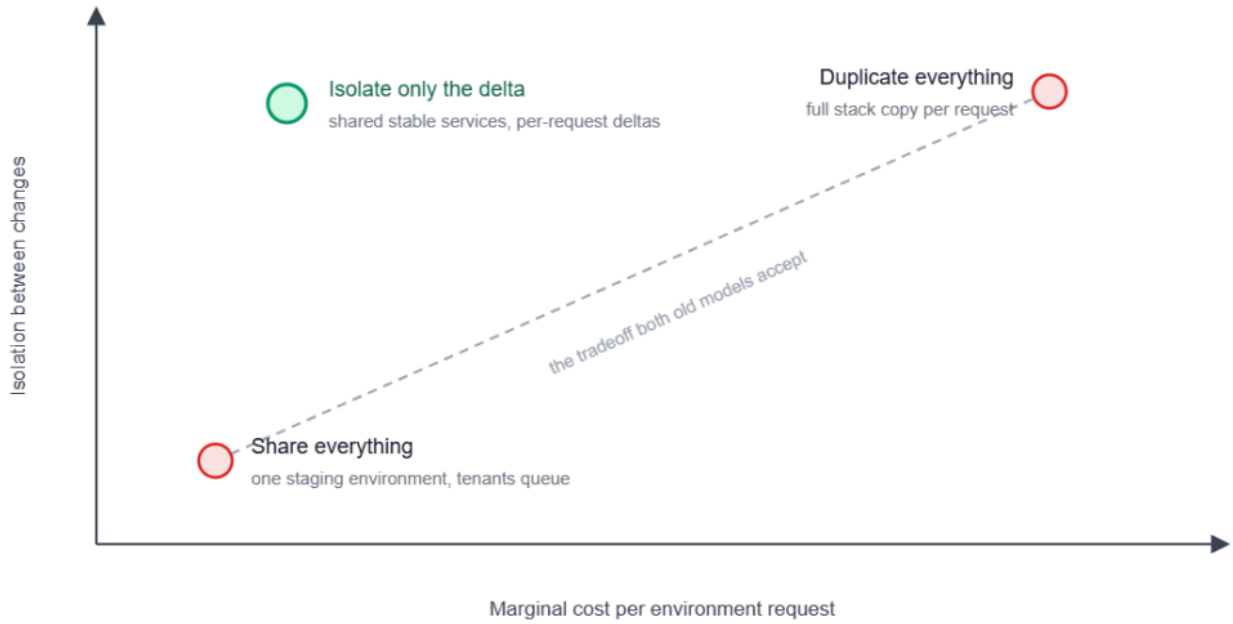
The shared model runs one staging environment and admits tenants in turn. Queueing theory has described this failure mode since 1961. Little's law says the number of requests in a system equals the arrival rate multiplied by time in the system, so as arrivals approach the rate the environment can absorb, wait times stop degrading gracefully and start exploding. Agents multiply the number of arrivals by 5-10 while the completion rate remains fixed.

Shared staging also fails on isolation. One broken change contaminates the environment for every tenant behind it, so the line does not merely lengthen; it periodically resets to zero while someone hunts down the offending commit.

ms respond to the wait the way people always respond to a slow shared resource, by batching. Changes pile into larger deployments, making each trip

queue worse.

Both models sit at the wrong ends of the same curve, paying full cost for full isolation or zero marginal cost for zero isolation. Neither is a point from which you can operate a serving system.



Environments are a serving system now

The mental model that fits this demand curve already exists inside every platform team. It is the one used for compute. A serving system is judged on latency, concurrency, marginal cost per request, and safe multi-tenancy on shared infrastructure, and those are exactly the four requirements agent-driven demand imposes on environments. A serving system is also something its clients invoke directly, through an interface rather than a person, which is the property that matters most once those clients are agents.

Renaming the problem matters because it changes who owns it and how it gets measured. A provisioning workflow is done when the environment exists. A serving system is never done. It has dashboards, capacity plans, and error budgets, and it is expected to absorb demand spikes without a human in the loop.

latency target drops from hours to seconds."

The mindset gap shows up on every operational dimension. The unit of work ceases to be a ticket and becomes a request. The latency target drops from hours to seconds. The success metric shifts from closed tickets to p99 latency at peak concurrency.

	Provisioning mindset	Serving mindset
Unit of work	An environment ticket	A request, like a query
Latency target	Minutes to hours	Seconds
Cost curve	Linear per environment	Near-zero marginal cost
Tenancy	One tenant per copy	Many tenants on shared infrastructure
Operations	Fulfilled, then forgotten	Run against SLOs
Success metric	Tickets closed	p99 latency, concurrency, cost per request

Serve the delta, not the whole stack

One architecture meets all four serving requirements by refusing to copy anything that has not changed. Run a single high-fidelity, stable copy of the system, deployed continuously from main. When a validation request arrives, deploy only the services that changed as lightweight, ephemeral environments, and route that request's traffic through their own versions, while everything else falls through to the shared, stable environment.

Each serving property follows from the delta. Latency lands in seconds because starting one or two services is fast. Marginal cost approaches zero because tenants share the stable environment. Concurrency is bounded by cluster capacity rather than by environment count, and isolation holds because each request sees only its own changed services, not anyone else's.

between refreshes. Sharing one stable copy that is continuously deployed from main gives every validation request the same real, current dependencies without reproducing them per request.

Routing is the implementation detail rather than the point. Service meshes can carry the routing label, sidecar-free approaches can too, and propagating a label through a call chain is a solved problem in most modern stacks. This is the pattern Signadot enables off-the-shelf.

Agents provision their own environments

An environment that arrives in seconds and costs almost nothing is not only fast enough to keep up with agents. It is cheap and fast enough for them to operate. When requesting one is an API call rather than a ticket, provisioning becomes a step within the agent's own loop: ask for an environment, deploy the change to it, run the checks, read the result, tear it down, and repeat in the next iteration.

Both properties are what make that possible. A workflow measured in minutes and gated on human approval can never fit within a build-test-fix cycle, because the agent would spend its run waiting in a queue it cannot influence. Near-zero marginal cost makes a discarded environment a non-event, and seconds of latency lets validation live inside the loop instead of after it. Once the environment is something an agent requests for itself, the human stops being the rate limiter, and the platform's serving capacity takes over.

Validation throughput is what ships AI code

Agents made generation cheap and pushed the bottleneck downstream, onto whether a change can be validated as fast as it is written. Validation throughput, not lines generated, now decides how much AI-written code actually ships, and it is a property of your platform rather than any model.

"Validation throughput, not lines generated, now decides how much AI-written code actually ships."

---

at environments as a serving system, and environment capacity becomes a ıension you plan and budget like compute or continuous integration (CI) runners.

paths for people.

The next job is self-service for developers and agents that can scale with agent-driven velocity, and we built Signadot for exactly that.

TNS

Arjun Iyer, CEO of Signadot, is a seasoned expert in the cloud native realm with a deep passion for enhancing the developer experience. Boasting over 25 years of industry experience, Arjun has a rich history of developing internet-scale software and...

Read more from Arjun Iyer →

Learn More →

Kubernetes Staging Environments: How to Build, Run, and Share One
17 July 2026

Catch Performance Regressions Before the Pull Request With k6 and Signadot Plans
16 July 2026

Comparing Kubernetes Development Tools in 2026: Telepresence, mirrord, Okteto, and Signadot
18 June 2026

Watch Claude Code Validate a Microservices Change in a Closed Loop
4 June 2026

Signadot Plans
20 May 2026

AI Code Testing for Cloud-Native Systems: Why Unit Tests Aren't Enough
8 May 2026

SUBSCRIBE

ARCHITECTURE

Cloud Native Ecosystem
Containers
Databases
Edge Computing
Infrastructure as Code
Linux
Microservices
Open Source
Networking
Storage

ENGINEERING

AI
AI Engineering
API Management
Backend development
Data
Frontend Development
Large Language Models
Security
Software Development
WebAssembly

OPERATIONS

AI Operations
CI/CD
Cloud Services
DevOps
Kubernetes
Observability
Operations
Platform Engineering

CHANNELS

Podcasts
Ebooks
Events
Webinars
Newsletter
TNS RSS Feeds

THE NEW STACK

About / Contact
Sponsors

Frontend Developer Roadmap
Backend Developer Roadmap
Devops Roadmap

FOLLOW TNS